

Rules as Code: A Survey

July 2026

Rules as Code: A Survey

Compiled July 2026 from public web sources. Sources are linked inline and listed at the end.

1. What is Rules as Code?

Rules as Code (RaC) is the idea that governments should produce an official, machine-consumable version of their rules — legislation, regulation, and policy — alongside the natural-language version, rather than leaving every agency, vendor, and citizen to re-interpret the text independently. The encoded version does not replace the law; it is a parallel artifact designed to be executed by software, so that digital services (benefits eligibility checkers, tax calculators, permit systems) implement the *same* interpretation of the rules ([Salsa Digital](#), [OECD](#)).

The canonical framing comes from the OECD Observatory of Public Sector Innovation’s 2020 working paper “*Cracking the Code: Rulemaking for humans and machines*”, which argues that RaC requires governments to “deliberately and explicitly rethink existing rulemaking processes” so that rules are drafted from the start to be both human- and machine-consumable — not merely translated into code after the fact.

The core claims

Proponents argue RaC can:

- **Reduce interpretation drift.** Today, a single statute is independently re-implemented by dozens of agencies and vendors, each embedding its own interpretation in opaque systems. A canonical encoding means implementation better matches legislative intent.
- **Improve the rules themselves.** The act of encoding surfaces ambiguities, gaps, and internal contradictions in draft legislation before it passes — encoding functions as a rigorous form of review.
- **Enable digital service delivery.** Executable rules power eligibility checkers, “tell us once” service portals, and automated calculations directly from the authoritative source.
- **Increase transparency.** A public, open-source encoding is inspectable in a way that a vendor’s proprietary benefits system is not.

2. Origins and global landscape

RaC emerged in the late 2010s from digital-government teams, notably:

- **New Zealand** — The Service Innovation Lab’s *Better Rules / Legislation as Code* program (2018–) pioneered multidisciplinary drafting teams (policy, legal, service design, software) producing human and machine versions of rules concurrently. Its outputs include **SmartStart**, a life-event portal for new parents, and the influential *Better Rules for Government* discovery report ([Service Innovation Lab](#)). A follow-on critical report for NZ decision-makers is Hamish Fraser’s *Legislation as Code* ([hamish.dev](#)).
- **France** — An early adopter via **OpenFisca**, an open-source platform originally built to model the French tax-benefit system, now the most widely used RaC engine internationally ([openfisca.org](#)).

- **Australia (NSW)** — NSW Government ran prominent RaC pilots (e.g., community gaming regulation encoded during drafting); TJ Harrop’s writing from this period is a widely cited practitioner account ([Medium](#)).
- **Canada** — The Canada School of Public Service and collaborators built **Blawx**, a visual, logic-programming-based encoding tool aimed at non-programmers; Jason Morris’s *Rules as Code Diary* documents this line of work ([Medium](#)).
- **Jersey** — Legislative drafter Matthew Waddington has advocated for RaC from inside the drafting profession, focusing on how drafters can write “code-friendly” law without eliminating necessary discretion ([Law School Policy Review interview](#)).
- **United States** — Activity has centered on public benefits: the Beeck Center’s **Digital Benefits Network** runs a Rules as Code Community of Practice and has pushed for standardized, machine-readable communication of benefits rules across federal/state/county layers ([Digital Government Hub](#)). Think tanks have also picked up the theme — e.g., a 2025 G7/T7 task-force paper argues RaC can reduce compliance friction in the global economy ([CIGI](#)).

3. Tools and technical approaches

Three broad technical families dominate:

Family	Representative tools	Character
Imperative microsimulation libraries	OpenFisca (Python), PolicyEngine	Rules encoded as parameterized formulas over entities (person, household). Mature, production-proven for tax/benefit calculation; oriented to programmers rather than drafters.
Declarative / logic programming	Blawx (s(CASP) + Blockly visual UI), Prolog-family encodings, Legalese	Rules as logical predicates; supports explanation (“why/why not”) and non-monotonic reasoning (defaults and exceptions), closer to how law is structured. Blawx targets non-programmers (Salsa Digital , COHUBICOL).
Domain-specific languages for law	Catala (INRIA, France)	A DSL whose structure mirrors the statute (default logic, article-by-article annotation), designed for lawyer–programmer pair programming and formal verification.

A related strand is **drafting-time syntax**: proposals for legislative drafting conventions that make natural-language law easier to formalize — e.g., “*The Legislative Recipe: Syntax for machine-readable legislation*” ([arXiv](#)) and OECD-OPSI’s catalog of techniques for using legal encodings in the drafting room ([OPSI](#)).

4. Standing challenges and critiques

The critical literature converges on a few themes:

1. **Open texture and discretion.** Following H.L.A. Hart, natural-language law is deliberately open-textured: terms like “reasonable” or “suitable accommodation” defer judgment to future decision-makers. Deterministic encodings either cannot capture this or capture it by silently resolving it —

converting discretion into a fixed policy choice made by a programmer ([Springer, *The challenge of open-texture in law*](#)). Waddington's response is that large parts of law (rates, thresholds, date arithmetic, eligibility conjunctions) are *not* open-textured and are safe to encode, and that drafters can flag discretionary elements explicitly.

2. **Interpretation is not neutral.** Encoding *is* interpretation. Who has authority to say the code correctly captures the statute? If the encoded version diverges, which governs? Scaling this up across a whole statute book multiplies the problem ([ScienceDirect, *Representing legislative Rules as Code: reducing the problems of 'scaling up'*](#)).
3. **Cost and maintenance.** Manual encoding is slow, expert-intensive, and must track every amendment. This — not conceptual objections — has been the practical brake on RaC adoption, and it is precisely the pain point AI is now being aimed at.
4. **Accountability and black boxes.** Where encodings are proprietary (as most vendor benefits systems are), automation *reduces* transparency. RaC advocates answer with open source, but the risk cuts both ways ([Interoperable Europe](#)).
5. **Automation-bias risk.** Once rules run automatically at scale, errors also run automatically at scale (Australia's Robodebt is the standard cautionary tale cited across this literature — an example of automated *mis*-encoding of the law, not of RaC done properly).

5. The AI opportunity

The arrival of capable large language models has reshaped the RaC conversation. Manual encoding was RaC's biggest bottleneck; LLMs attack exactly that bottleneck — while introducing a new tension, since LLMs are probabilistic and RaC's whole value proposition is determinism.

5.1 AI as encoder: policy-to-code translation

The most direct opportunity: use LLMs to draft the encoding, with humans shifting from *programmers* to *reviewers*.

The most substantial public evidence is the Beeck Center's **Policy2Code Prototyping Challenge** and the resulting report *AI-Powered Rules as Code: Experiments with Public Benefits Policy* (2024–25). Teams used commercial LLMs — via direct prompting, RAG, and fine-tuning — to translate SNAP and Medicaid eligibility rules across seven states into plain-language summaries, pseudocode, and runnable code ([Beeck Center, summary](#)). Headline finding: **LLMs can meaningfully support policy-to-code translation, but for any policy with complex logic they require external knowledge (RAG over authoritative sources) and human oversight in an iterative loop** — reviewer-in-the-loop, not autonomous encoding.

Implications if this matures:

- **Cost collapse for encoding.** The scaling-up problem (challenge #3 above) becomes tractable; whole benefit programs or tax codes could get first-draft encodings in days.
- **De-duplication across government.** In federal systems, the same federal rule is re-implemented by every state; AI-assisted encoding plus shared repositories could eliminate that duplication.
- **Legacy modernization.** The same translation capability works in reverse and sideways: LLMs can lift rules *out* of legacy COBOL/vendor systems and re-express them in maintainable RaC form — arguably a bigger near-term market than encoding new law.

5.2 AI + RaC as a hybrid (neurosymbolic) architecture

A second framing treats RaC and LLMs as complementary halves of one system: the LLM handles natural language (understanding a citizen's messy situation, extracting facts, explaining results in plain language), while the symbolic RaC engine does the actual legal computation — deterministic, auditable, correct by construction.

- The **MIT Computational Law Report** piece *Governing Digital Legal Systems* frames this as the central design question: RaC reasons deductively, LLMs inductively, and naive substitution of one for the other fails; governance must assign each layer the role it is reliable at (law.mit.edu).
- A concrete demonstration: Canada’s Public Health Agency built a **conversational tool over a Blawx encoding of a privacy statute** — the LLM converses, the logic engine answers, and every answer carries a formal justification tree ([GitHub demo](#)).
- The academic literature has moved fast here. Recent work couples LLMs with Prolog, SAT/ASP solvers, and legal DSLs so that the LLM *translates* facts and rules into symbolic form and a solver does the reasoning ([Towards Robust Legal Reasoning](#), [Neuro-Symbolic Offloading for legal adjudication](#), [ICLR 2026 workshop work on neuro-symbolic judgment prediction](#)).
- Evaluation is catching up: benchmarks now test whether LLMs are genuinely reasoning or pattern-matching in statutory domains — e.g., contamination-aware evaluation in tax law ([Reasoners or Translators?](#)), German subsumption reasoning ([BenGER](#)), and faithfulness of LLMs as *autoformalizers* of legal rules ([Know Your Limits](#)). A consistent result: LLM-as-solver is brittle; LLM-as-formalizer feeding a verified engine is markedly more robust — which is, in effect, an empirical argument *for* Rules as Code.

5.3 AI as verifier and drafting assistant

- **Formal verification as a feedback signal.** A 2026 arXiv paper proposes using formally verified encodings of law as a *reward signal* for legal AI — the RaC artifact becomes the ground truth against which model outputs are automatically checked, closing the loop between encoding and model improvement ([Closing the Loop](#)).
- **Legislative quality assurance.** LLMs can cross-reference statutes against their implementing regulations, flag conflicts, redundancies, and orphaned provisions — extending RaC’s “encoding as review” benefit without full encoding ([POPVOX Foundation](#), [Harvard Journal on Legislation](#)).
- **Drafting-room copilots.** Combining §3’s drafting-time techniques with LLMs: as a drafter writes, an assistant maintains a parallel formalization and immediately surfaces ambiguities, undefined terms, and unreachable conditions — making “encode while drafting” cheap enough to be routine.

5.4 AI as the interface to encoded rules

Even a perfect encoding is useless to a citizen who can’t operate it. LLMs solve RaC’s last-mile problem: conversational front-ends that gather a person’s facts in natural language, run them against the authoritative encoding, and explain the result — with the *answer* guaranteed by the symbolic engine rather than the model. This pattern (LLM interface, RaC oracle) appears across the PHAC demo, the Beeck Center work, and commentary like The Fulcrum’s “*When rules can be code, they should be*” ([The Fulcrum](#)).

5.5 New risks AI introduces

The literature is equally clear about the hazards:

- **Plausible-but-wrong encodings.** LLM-generated code fails in ways that read as correct; a subtly wrong eligibility encoding deployed at scale is a Robodebt-class failure. Every serious study insists on human legal review and test suites derived from authoritative examples.
- **Laundering interpretation through a model.** Encoding is interpretation (challenge #2); AI encoding is interpretation by an unaccountable statistical process. Provenance — which model, which prompt, which human approved which clause — becomes a governance requirement.
- **The determinism–probabilism boundary must be explicit.** The Law School Policy Review analysis stresses that letting the LLM answer legal questions directly (rather than route them to the encoding) reintroduces the black-box problem RaC was meant to solve; mitigations like chain-of-thought and interpretable sub-models help but don’t eliminate it ([LSPR](#)).
- **Discretion still doesn’t compile.** AI makes it cheaper to encode more, which increases the temptation to encode what shouldn’t be encoded. Open-textured provisions need to be *flagged and routed to*

humans, and LLMs may actually help here — classifying provisions by encodability rather than forcing everything into code.

6. Outlook

The field's trajectory can be summarized in three phases:

1. **2018–2022 — proof of concept.** NZ, France, NSW, Canada showed RaC works in pilots; OECD gave it a policy framework; adoption stalled on encoding cost and institutional inertia.
2. **2023–2025 — the LLM inflection.** Policy2Code and academic neurosymbolic work reframed the bottleneck: encoding becomes AI-drafted and human-verified, and RaC engines find a second life as the *symbolic backbone* that makes government AI trustworthy.
3. **2026 onward — convergence.** The most credible near-term architecture is now widely agreed: **LLMs at the edges (translation, interface, review), verified rule engines at the core (computation, guarantees), humans at the points of interpretation and accountability.** The open questions are institutional, not technical: who certifies an encoding as official, how amendments propagate, and how discretion is preserved.

The deeper point several authors make: AI doesn't just accelerate Rules as Code — it strengthens the case for it. As governments deploy LLMs anyway, an authoritative, testable, machine-consumable statement of the rules is the single best control for keeping those systems honest.

Sources

- [OECD OPSI — Cracking the Code: Rulemaking for humans and machines \(WP No. 42\)](#)
- [OECD OPSI — About Rules-as-Code \(whitepaper attachment\)](#)
- [Salsa Digital — What is Rules as Code? · Rules as Code · Rules as Code and Blawx](#)
- [NZ Service Innovation Lab — Better Rules and Legislation as Code](#)
- [Hamish Fraser — Legislation as Code \(report for NZ decision-makers\)](#)
- [TJ Harrop — What are 'machine readable laws'?](#)
- [Jason Morris — Rules as Code Diary, 2020–2021 in Review](#)
- [OpenFisca · OpenFisca documentation](#)
- [COHUBICOL — Blawx typology entry](#)
- [PHAC — Privacy Rules-as-Code conversational demo \(GitHub\)](#)
- [arXiv — The Legislative Recipe: Syntax for machine-readable legislation](#)
- [OECD OPSI — New techniques for building and using legal encodings in the drafting room](#)
- [ScienceDirect — Representing legislative Rules as Code: reducing the problems of 'scaling up'](#)
- [Springer AI & Law — The challenge of open-texture in law](#)
- [Law School Policy Review — Rules as Code and Large Language Models · Conversation with Matthew Waddington](#)
- [MIT Computational Law Report — Governing Digital Legal Systems](#)
- [Beeck Center — AI-Powered Rules as Code: Experiments with Public Benefits Policy · Summary at Digital Government Hub](#)
- [Digital Government Hub — Digitizing Policy + Rules as Code topic hub · Exploring Rules Communication for U.S. public benefits](#)
- [CIGI / T7 — Rules as Code for a More Transparent and Efficient Global Economy](#)
- [POPVOX Foundation — Unlocking the Code: How legislators can use AI to demystify regulation](#)
- [Harvard Journal on Legislation — First principles on LLM capabilities and federal rulemaking](#)
- [The Fulcrum — When rules can be code, they should be](#)
- [Interoperable Europe — Rules as Code: an open approach](#)
- [arXiv \(neurosymbolic & evaluation\): Towards Robust Legal Reasoning · Neuro-Symbolic Offloading for legal adjudication · Reasoners or Translators? \(tax law\) · BenGER \(German law\) · Know Your Limits \(LLMs as autoformalizers\) · Closing the Loop: formally verified law as reward signal · OpenReview: neuro-symbolic legal judgment prediction](#)